Bonjour à tous :-)

Flags en C (universel)

Fichier : Lock et Dirty

int flags : 0 0 0 0 0 1 0 0 1 **0** 1

Def

```
#define F_LOCK   1 << 1
#define F_DIRTY  1 << 3
```

Set :
0 0 0 0 0 1 0 0 1 **0** 1
OR 0 0 0 0 0 0 0 0 0 **1** 0
0 0 0 0 0 1 0 0 1 **1** 1

```
flags ⊨ F_LOCK
flags ⊨ F_LOCK | F_DIRTY
```

Test :

0 0 0 0 0 1 0 0 1 **x** 1
ET 0 0 0 0 0 0 0 0 0 **1** 0
0 0 0 0 0 0 0 0 0 **?** 0

```
if ( flags & F_LOCK ) {  ....}
if ( flags & ( F_LOCK | F_DIRTY ) ) {  ....}
```

Unset :

0 0 0 0 0 1 0 0 1 **x** 1
ET 1 1 1 1 1111 1 **0** 1
0 0 0 0 0 1 0 0 1 **0** 1

```
flags &= ~ F_LOCK
flags &= ~ ( F_LOCK | F_DIRTY )
```

Syscall (pour l'UNIX V6)

```
// Implem d'un syscall sys_xxxx

sys_xxx ()
{
    // recup les param
    int param_a = u.u_arg[0];
    int param_b = u.u_arg[1];

    ......

    // retour erreur
    if ( param_a == 0 ) {
        u.u_error = EINVAL ;
        return ;
    }

    ......

    // retour val
    u.u_ar0[R0] = val ;  // ⟺ return val du monde S → monde U
}
```

Pour tout process (UNIX V6)

```
// &proc[0] ⟺ proc  mais avec du typage

// Pour tous les process
for ( p = &proc[0] ; p < &proc[NPROC] ; p++) {
    if ( p->p_stat == NULL )
        continue ;
    .....
}

// Cherche la struct proc du process "pid"
for ( p = &proc[0] ; p < &proc[NPROC] ; p++) {
    if ( p->p_stat == NULL && p→p_pid ≠ pid )
        continue ;
    .....
}
```

Fonctions user des signaux

kill → ce n'est pas fait pour tuer
ipc pour envoyer un message

```
#include <signal.h>

int kill (int pid, int sig) : send sig au process pid

signal (int sig, @fonction) : def le handler a exec quand on recoit
sig

alarm (int nb_sec) : demander à recevoir un SIGALRM dans nb_sec
secondes
```

Question 1

```
void mon_handler ( int sig ) {
  printf("Bonjour\n") ;
  signal(SIGARLM, &mon_handler) ; // traitement d'un sig ⇒ retour
  defaut
  alarm(1);
}

void main (int argc, char **argv) {
  signal(SIGARLM, &mon_handler) ;   // mon_handler ⟺ &mon_handler
  alarm (1) ;

  while (1)
    sleep(3); // sort du sleep a la reception d'un sig
}
```

}

Question 3

```
// proc.h
struct proc {
  ...
  int p_sig ; // signaux reçus sous forme de flags
  ...
} proc[NPROC] ;

// user.h
struct user {
  ...
  u_signal[NSIG] // tableau pointeur de fonction : addr des handlers
  ...
} u ;
```

Question 5

MODE U | MODE S
libc | kernel

kill(pid, sig) ------⟶ kill (⇔ sys_kill) --⟶ psignal ( )
signal(pid, @f) ------⟶ ssig

|

| if (issig()) // → test si on a un sig pendant et efface sig ignoré
psig() // → placer handler dans pile user

Algo fsig

i=1

0110011000

000000000001

shift n

i=2

011001100

000000000001

shift n

i=3

01100110

000000000001

shift n n=4

0110011

000000000001

le 4em signal est pendant (c'est le plus petit

Algo issig

tanque reste des signaux pendants

1) cherche le plus petit

2) test si ignoré ? avec la valeur du bit de poid faible de u_signal
1 = 0000000001 → signal ignoré
0 = 0000000000 → handler par defaut → mort du processus

@handler = ????????????0 car une addresse de fonction c'est tj paire

Algo sendsing

pile user originale

main	main
f	f
g	g
read ← SP placer au syscall dans u_ar0[SP]	read
handler ← SP restauré au retour mode U	

PC user original

@ d'une instruction dans read (ap TRAP)	@ premiere instruction du handler

ATTENTION : 1 signal seulement au retour en mode U
- si plusieurs signaux sont pendants, on empile que le handler du plus petit signal
- le reste au prochain passage de S vers U (prochain syscall, prochaine IT, ...)